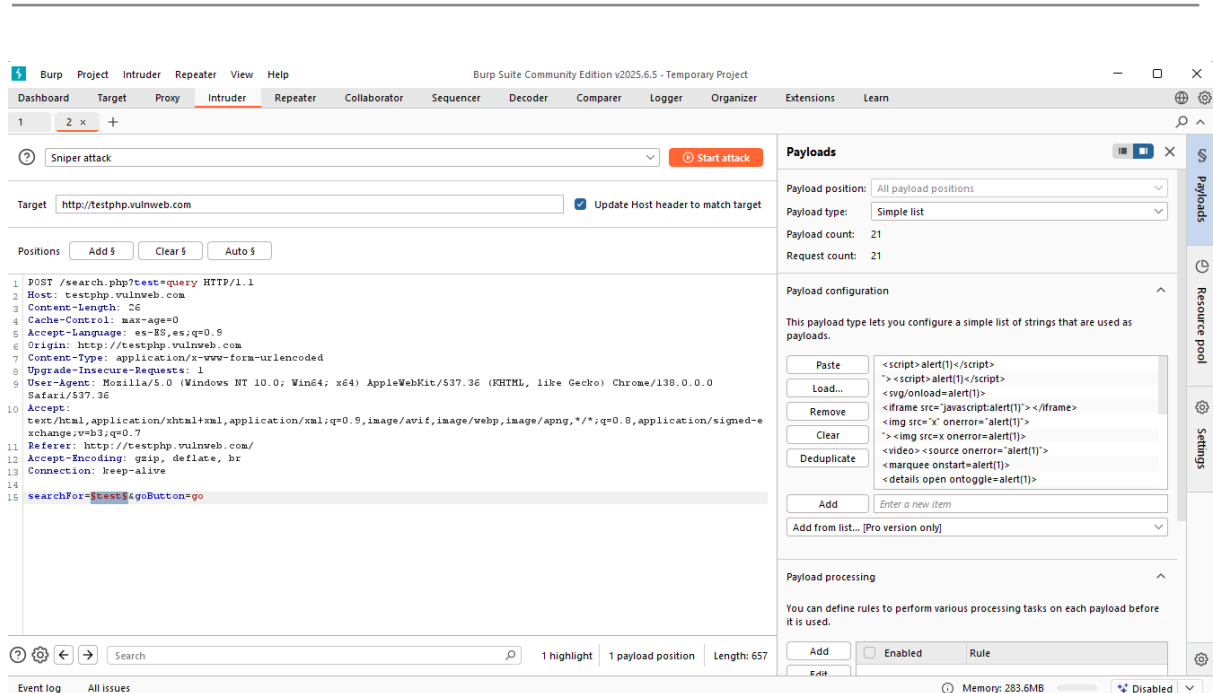
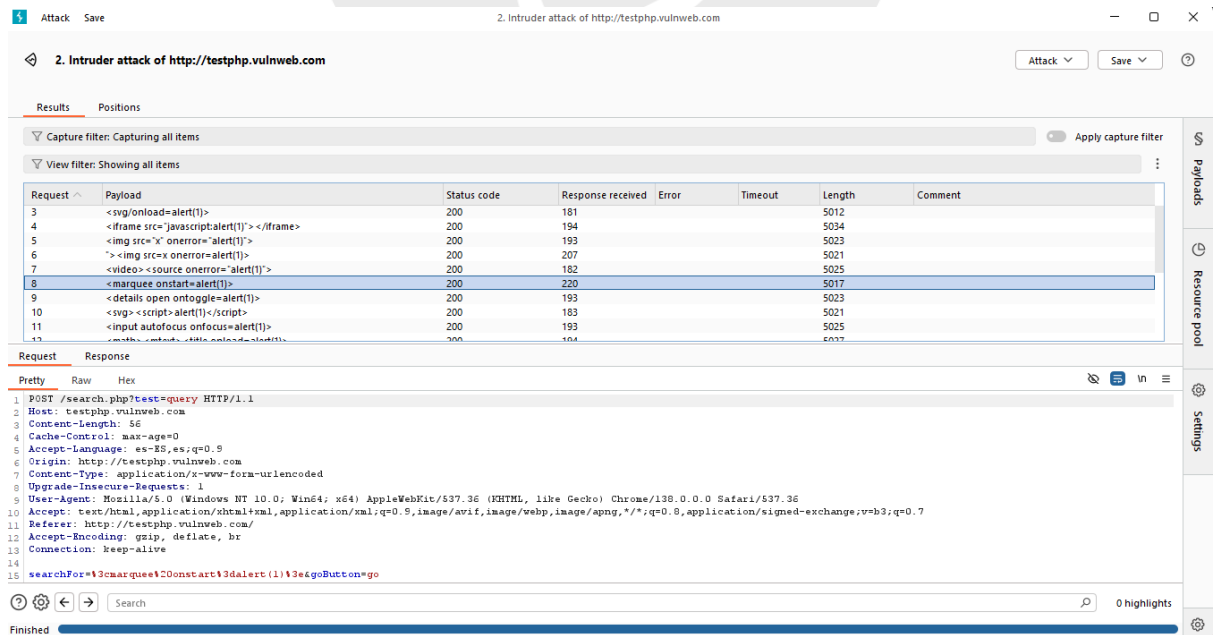


Análisis de vulnerabilidad XSS Reflejado con Burp Suite



The screenshot shows the Burp Suite Intruder tab for a 'Sniper attack' on the target 'http://testphp.vulnweb.com'. The 'Payloads' panel on the right is configured with a 'Simple list' of 21 payloads. The 'Payload configuration' section lists various XSS payloads, including alert boxes, SVG onload events, and iframe src attributes. The 'Payload processing' section is currently empty. The main window displays the HTTP request details for a POST to /search.php, showing headers like 'Host: testphp.vulnweb.com' and 'User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36'. The request body shows a search form with the value 'test&goButton=go'.

Configuración del ataque Intruder en Burp Suite



The screenshot shows the results of the Intruder attack. The 'Results' tab is active, displaying a table of 11 requests. The table columns are Request, Payload, Status code, Response received, Error, Timeout, Length, and Comment. The first request (index 3) shows a successful status code of 200. The 'Request' tab is also visible, showing the raw HTTP request details, including the headers and the body with the search form data.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
3	<svg/onload=alert()>	200	181			5012	
4	<iframe src='javascript:alert()'></iframe>	200	194			5034	
5		200	193			5023	
6		200	207			5021	
7	<video><source onerror=alert()>	200	182			5025	
8	<marquee onstart=alert()>	200	220			5017	
9	<details open ontoggle=alert()>	200	193			5023	
10	<svg><script>alert()</script>	200	183			5021	
11	<input autofocus onfocus=alert()>	200	193			5025	
12	$alert(1)$	200	184			5017	

Resultados de Intruder

✓ 3. Explicación técnica del hallazgo

💬 Análisis:

El sitio web acepta entradas del usuario sin sanitizar ni escapar los caracteres especiales del HTML, como `<`, `>`, `"`, `'`. Esto permite inyectar etiquetas como `<script>` y atributos como `onerror`, los cuales el navegador ejecuta como código JavaScript cuando se reflejan en la respuesta del servidor.

Ejemplo de payload exitoso reflejado:

```
<script>alert(1)</script>
```

Dado que **la respuesta HTTP fue 200** y el contenido tuvo un cambio en el tamaño, es altamente probable que el payload se haya incluido en el HTML generado, lo que representa un riesgo para el usuario.

4. Recomendaciones de mitigación

🔒 Para el equipo de desarrollo:

1. **Escapar correctamente la salida:** Nunca renderizar directamente en HTML valores ingresados por el usuario.
 - Usar funciones como `html.escape()` o equivalentes del framework.
2. **Validar la entrada:** Aplicar filtros estrictos al input, validando solo lo necesario (whitelisting).
3. **Usar Content Security Policy (CSP):**

Agregar headers como:

```
Content-Security-Policy: default-src 'self'; script-src 'self';
```

○

4. **Evitar `innerHTML` o `document.write()`** en el frontend.
5. **Aplicar frameworks que escaparán automáticamente la salida**, como React, Angular, etc.

¿Por qué los ataques XSS reflejados son peligrosos para los usuarios finales?

Porque permiten al atacante inyectar y ejecutar scripts maliciosos directamente en el navegador del usuario. Esto puede llevar a:

- Robo de cookies o tokens de sesión.
- Redirección a sitios falsos (phishing).
- Registro de pulsaciones de teclas (keyloggers).
- Manipulación de la interfaz del sitio para engañar al usuario.

¿Qué importancia tiene codificar correctamente la salida en aplicaciones web?

Codificar correctamente (escaping) asegura que los caracteres especiales del input **no sean interpretados como código HTML o JavaScript**, sino como texto plano. Esto previene que un atacante inyecte código malicioso y lo ejecute en el navegador del usuario.

¿Qué limitaciones tiene el análisis automático frente al análisis manual?

Análisis Automático	Análisis Manual
✓ Rápido	✓ Más preciso
✓ Repetible	✓ Detecta lógica compleja
✗ Puede omitir vectores complejos	✗ Más lento
✗ No siempre interpreta comportamiento dinámico del DOM	✓ Puede adaptarse al flujo de la aplicación